

AI Programming Foundations Project

A reproducible exploratory workflow for the Titanic passenger dataset

Johannes Pawelczyk

Module summary | 28 September 2026

Project objective

Build a transparent Python workflow that ingests, cleans, explores, visualizes, and communicates a real tabular dataset without training a model.

1 Overview

This project builds an end-to-end exploratory data workflow in Python using NumPy, Pandas, Matplotlib, Seaborn, and Jupyter. The submitted notebook loads the Titanic training dataset, audits and cleans its records, computes grouped summaries, produces three labeled figures, and documents assumptions and limitations. The dataset is available from the Titanic - Machine Learning from Disaster project page.

The workflow was designed for rerunning rather than as a one-off analysis. Danchev (2022) presents open-source Python, Jupyter notebooks, Markdown, and real-world datasets as an integrated basis for transparent, reproducible, and ethical data analysis. Following that principle, this project keeps code, narrative, outputs, dependency versions, source data, and figures together.

2 Dataset Description

The CSV contains 891 passenger observations and 12 source variables. Key fields include survival status, passenger class, recorded sex, age, family relations aboard, fare, cabin, and embarkation port. Survival is binary: 342 passengers survived and 549 did not, for an observed survival rate of 38.4%.

Quality check	Observed value	Workflow response
Age missing	177 (19.9%)	Group-median imputation + indicator
Cabin missing	687 (77.1%)	Retain only cabin-known indicator
Embarked missing	2 (0.2%)	Fill with modal port S
Exact duplicates	0	Checked; none removed

The analysis focuses on survival rates across passenger class, recorded sex, age, and derived family size. The file is historical and observational, so the analysis describes associations and does not interpret these attributes as causal mechanisms or normative categories.

3 Workflow Description

1	Ingestion	Load the local CSV; show first rows, dimensions, types, and missingness.
2	Cleaning	Normalize names; preserve missingness; impute age; derive transparent features.
3	EDA	Compute passenger counts, survivor counts, and survival rates by group.
4	Visualization	Create three labeled plots and save reproducible PNG outputs.
5	Summary	Interpret patterns with explicit assumptions, limitations, and next steps.

4 Key Decisions and Assumptions

Cleaning choices

The workflow defines and uses two documented cleaning functions. The first standardizes column names. The second removes exact duplicates, trims text, tracks missing age, fills age with the median of each sex/class group, fills two missing embarkation values with the modal port, replaces sparse cabin strings with a cabin-known indicator, and derives family size and solo-travel status.

This strategy keeps all 891 rows without pretending that missing values were observed. Wickham (2014) argues that a consistent tabular structure - one variable per column and one observation per row - makes data easier to manipulate, model, and visualize. The cleaned table follows that structure and avoids storing multiple meanings in a single field.

EDA and plot design

The exploratory function reports group size, survivor count, and survival rate together. Counts are essential because rates for large family sizes are based on very few passengers. Figure 1 emphasizes the strongest class/sex differences; Figure 2 shows distribution overlap rather than only means; Figure 3 pairs every rate with its group size. Percent axes are bounded at 0-100% to avoid exaggerated visual differences.

Assumptions

The workflow assumes the Kaggle file accurately transcribes the passenger records, that ``SibSp + Parch + 1`` is a useful family-size proxy, and that group-median age imputation is acceptable for descriptive plots. Imputed ages are not recovered facts; the ``age_missing`` indicator is retained so later work can test sensitivity to that choice.

5 Results and Interpretation

The overall observed survival rate is 38.4%. The figures below correspond directly to the labeled outputs in ``data_workflow.ipynb``.

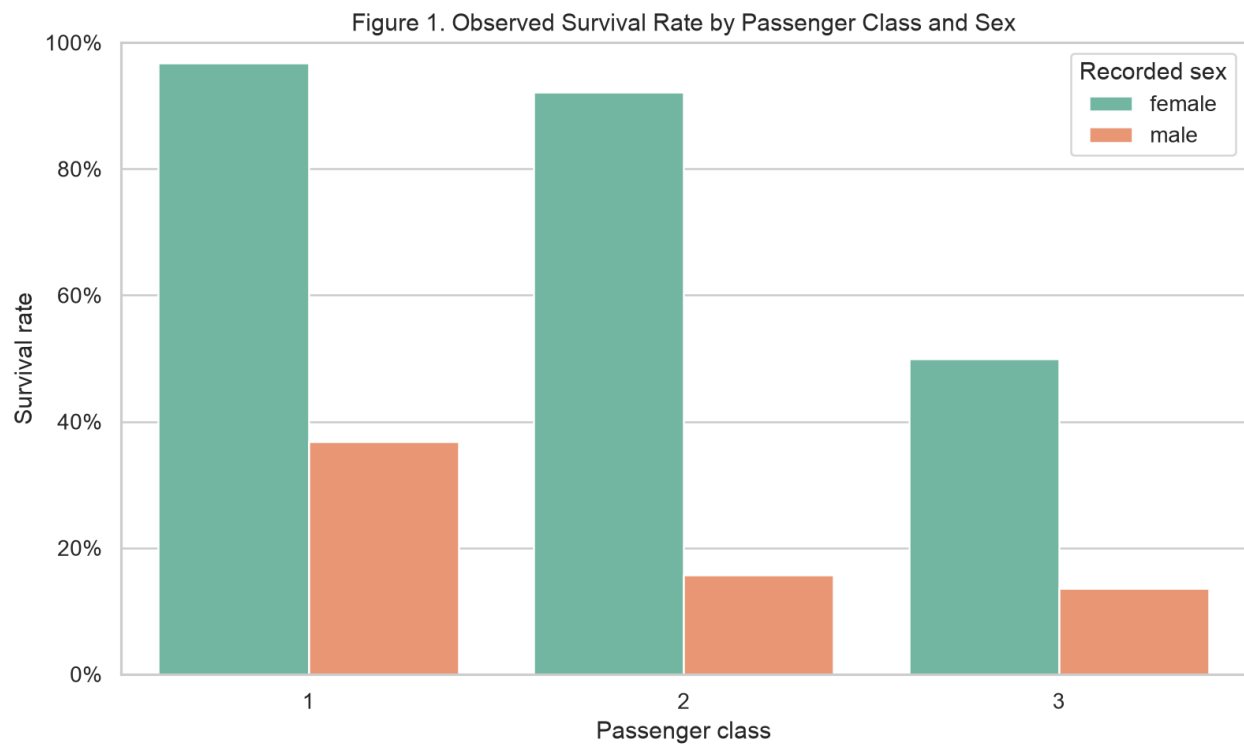


Figure 1. Recorded women had higher observed survival in every class. First-class women had the highest rate (96.8%); third-class men had the lowest (13.5%).

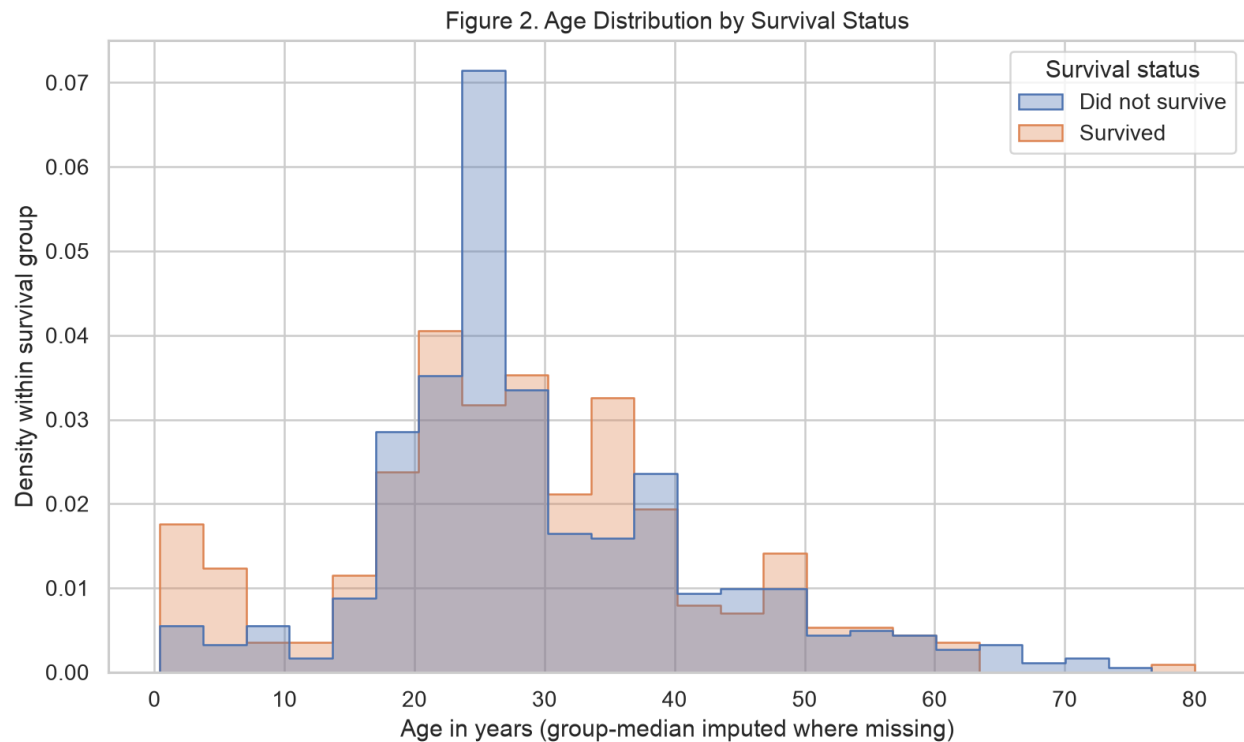


Figure 2. The age distributions overlap substantially. Both raw groups have a median age of about 28 years, while the child range contains a visible survivor peak. Because 177 ages were imputed for plotting, fine-grained differences remain uncertain.

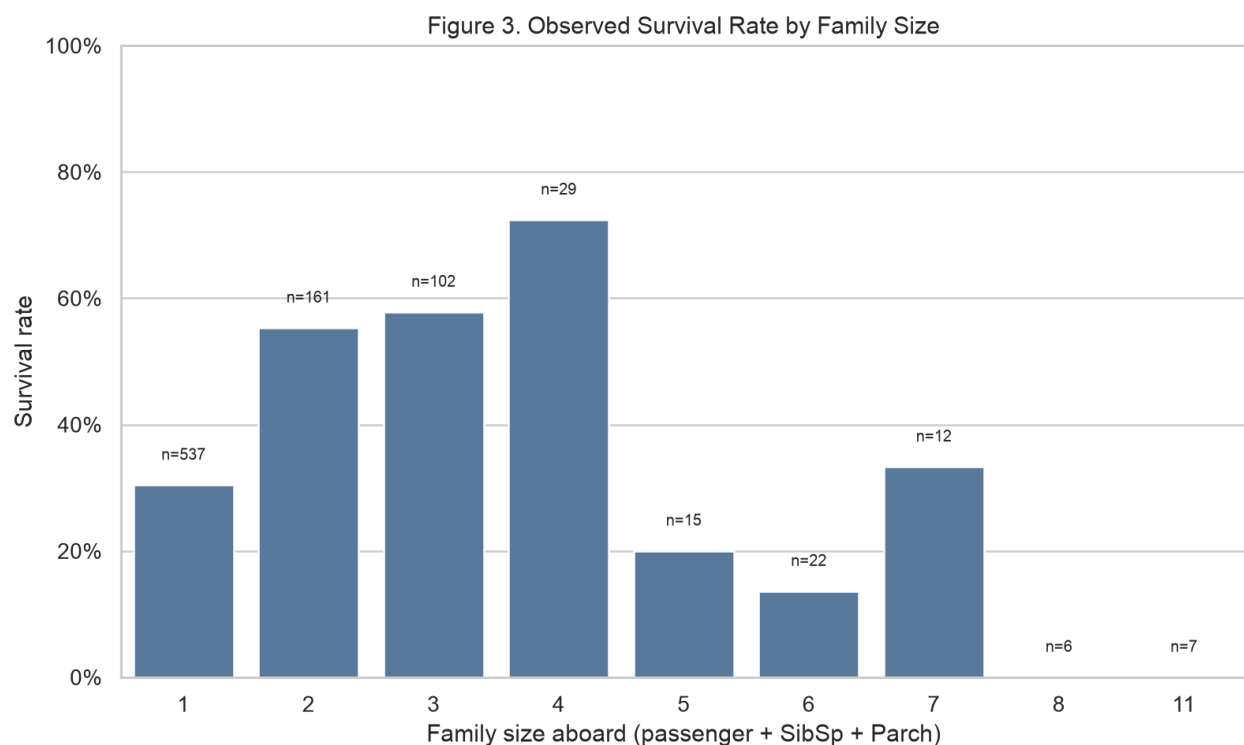


Figure 3. Families of two to four show higher observed survival than solo travelers. The sharp rates for family sizes 8 and 11 are based on only 6 and 7 passengers, so they should not support broad claims.

Taken together, the plots show that class and recorded sex separate outcomes more clearly than age alone, while family size has a non-linear association. One surprising result is that similar age medians coexist with local distribution differences. That contrast demonstrates why summary statistics and plots should be read together.

6 Responsible Practice: Bias and Data Quality

Cleaning can create bias when missingness is uneven across the same social groups being compared. Deleting every row with missing Cabin would remove 77.1% of the data; deleting missing ages would remove 19.9%. Either choice could change class and sex distributions. The workflow therefore retains all passengers, preserves missingness indicators, reports group counts, and avoids category-level claims about Cabin.

Group-median age imputation can still reduce variation and reproduce existing group structure. A stronger follow-up would compare complete-case results, multiple imputation, and models that use the missingness indicator. The historical labels also reflect a disaster and unequal social conditions. Any predictive extension would need subgroup evaluation, leakage controls, explicit justification for using sensitive attributes, and human review before deployment.

7 Reproducibility

The repository contains the source CSV, executed notebook, generated figures, pinned `requirements.txt`, concise run instructions, and this report. Relative paths let a reviewer run the project from the repository root. Every code cell has been executed from top to bottom without errors, and validation assertions check row count, required columns, resolved age/embarkation values, valid survival labels, and family-size bounds.

Version control records separate scaffold, analysis, and reporting stages. Work was performed on a ``development`` branch and integrated into ``main``, leaving both the history and the additional branch visible. This supports the transparent workflow that Danchev (2022) advocates: another analyst can inspect not only the final results but also the environment, decisions, and evolution of the work.

8 Sources and Citations

Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education*, 5(56), 156. <https://doi.org/10.21105/jose.00156>

Kaggle. (n.d.). *Titanic - Machine Learning from Disaster: Data*. Retrieved September 28, 2026, from <https://www.kaggle.com/c/titanic/data>

Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1-23. <https://doi.org/10.18637/jss.v059.i10>